

PRACTICAL 8:

```
nirvighna@DESKTOP-VS57D2  ×  +  ▾
GNU nano 7.2                memory_allocation.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;
    int n = 5;

    /* malloc() */
    int *arr = (int *)malloc(n * sizeof(int));

    if (arr == NULL)
    {
        printf("malloc failed\n");
        return 1;
    }

    printf("Memory allocated using malloc()\n");

    for (i = 0; i < n; i++)
    {
        arr[i] = (i + 1) * 10;
    }

    printf("Values after malloc:\n");

    for (i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    printf("\n");

    /* realloc() */
    n = 10;

    arr = (int *)realloc(arr, n * sizeof(int));

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```
nirvighna@DESKTOP-VS57D2I:~/practical8$ ./memory_allocation
Memory allocated using malloc()
Values after malloc:
10 20 30 40 50

Memory resized using realloc()
Values after realloc:
10 20 30 40 50 60 70 80 90 100

Memory freed using free()

Memory allocated using calloc():
0 0 0 0 0
calloc memory freed successfully
nirvighna@DESKTOP-VS57D2I:~/practical8$ |
```

```
nirvighna@DESKTOP-VS57D2  ×  +  v
GNU nano 7.2 memory_leak.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *arr;

    arr = malloc(10 * sizeof(int));

    if (arr == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 10; i++)
    {
        arr[i] = i + 1;
    }

    printf("Array values:\n");

    for (int i = 0; i < 10; i++)
    {
        printf("%d ", arr[i]);
    }

    printf("\n");

    /*
     * Intentionally not calling free(arr).
     * This creates a memory leak.
     */

    return 0;
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify

nirvighna@DESKTOP-VS57D2I:~/practical8$ ./memory_leak
Array values:
1 2 3 4 5 6 7 8 9 10
```

```
nirvighna@DESKTOP-VS57D2I:~/practical8$ valgrind --leak-check=full ./memory_leak
==1391== Memcheck, a memory error detector
==1391== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1391== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1391== Command: ./memory_leak
==1391==
Array values:
1 2 3 4 5 6 7 8 9 10
==1391==
==1391== HEAP SUMMARY:
==1391==     in use at exit: 40 bytes in 1 blocks
==1391==   total heap usage: 2 allocs, 1 frees, 1,064 bytes allocated
==1391==
==1391== 40 bytes in 1 blocks are definitely lost in loss record 1 of 1
==1391==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==1391==    by 0x1091BE: main (memory_leak.c:8)
==1391==
==1391== LEAK SUMMARY:
==1391==    definitely lost: 40 bytes in 1 blocks
==1391==    indirectly lost: 0 bytes in 0 blocks
==1391==    possibly lost: 0 bytes in 0 blocks
==1391==    still reachable: 0 bytes in 0 blocks
==1391==    suppressed: 0 bytes in 0 blocks
==1391==
==1391== For lists of detected and suppressed errors, rerun with: -s
==1391== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
nirvighna@DESKTOP-VS57D2I:~/practical8$ |
```